MEMORIA PRACTICA 1

01/10/2026
Sistemas Distribuidos

Eduardo Guerrero
816106

Índice

Introducción 3
Ejercicio 1 3
Ejercicio 2 4
Ejercicio 3 5

Introducción

Objetivos

En esta práctica se desarrollan 3 ejecuciones para distintivos objetivos:

- Analizar las latencias de los dos protocolos básicos de red (TCP y UDP) en las máquinas virtuales.
- Diseñar e implementar un algoritmo básico de sincronización de barrera distribuida.
- Familiarizarse con las arquitecturas cliente-servidor concurrente y máster-worker, aplicándolas en un ejercicio práctico.

Todo esto esta desarrollado en el lenguaje de programación Go

Recursos compartidos

Recurso	**Descripción**
Cluster	Raspberry Pi 4 Model B del Laboratorio 1.02
Máquinas asignadas	192.168.3.5, 192.168.3.6, 192.168.3.7 y 192.168.3.8 (máquinas 5 a 8)
Puertos asignados	29070 a 29079

Ejercicio 1

Análisis de las prestaciones de red TCP y UDP

Para el primer ejercicio se pide realizar disantos escenarios para ver las diferentes prestaciones que dan los dos protocolos TCP y UDP.

Para este ejercicio contamos con un cliente y un servidor TCP y otro cliente y servidor UDP.

Se estudian 6 casos

- Protocolo TCP: Misma máquina, dial fallido (el servidor no está en marcha)
- Protocolo TCP: Misma máquina, dial correcto (servidor en marcha)
- Protocolo TCP: Distinta máquina, dial fallido (el servidor no está en marcha)
- Protocolo TCP: Distinta máquina, dial correcto (servidor en marcha)
- Protocolo UDP: Misma máquina, envío de una letra y recepción de la respuesta
- Protocolo UDP: Distinta máquina, envío de una letra y recepción de la respuesta

Resultados:

	Tiempos
TCP en la misma maquina	Sin servidor: 622,233 micro s Con servidor: 934,322 micro s
TCP en diferente maquina	Sin servidor: 943,253 micro s Con servidor: 1056,452 micro s
UDP misma máquina	438,233 micro s

UDP diferente máquina	1086,253 micro s

Explicación costes TCP

Se utiliza la llamada net.Dial("tcp", addr) que realiza 4 tareas:

1. Resolución de la dirección
2. Creación del socket
3. Inicio del establecimiento de conexión
4. Espera respuesta
5. Finaliza

- Con servidor: el destino responde con SYN-ACK, el cliente responde con ACK y la conexión queda establecida. El dial vuelve a recibir el SYN-ACK
- Sin servidor: la maquina esta encendida pero el servidor no. SYN con un segmento RST y Dial devuelve el error connection refused. El destino no crea la conexión. Por eso más lento.
- Diferencia misma maquina o distinta maquina: Desde la misma máquina el programa pasa por la interfaz *loopback* por lo que no intervienen la tarjeta de red. En cambio, a otra maquina tiene que hacer todo el proceso de conexión a estar otra máquina

Explicación costes UDP

UDP no tiene conexión, y DialUDP no envía ningún paquete: solo crea el socket.
No hay handshake, ni confirmaciones, ni retransmisiones: si el datagrama se perdiera, el ReadFrom quedaría bloqueado indefinidamente

Ejercicio 2

Sincronización de Barrera Distribuida

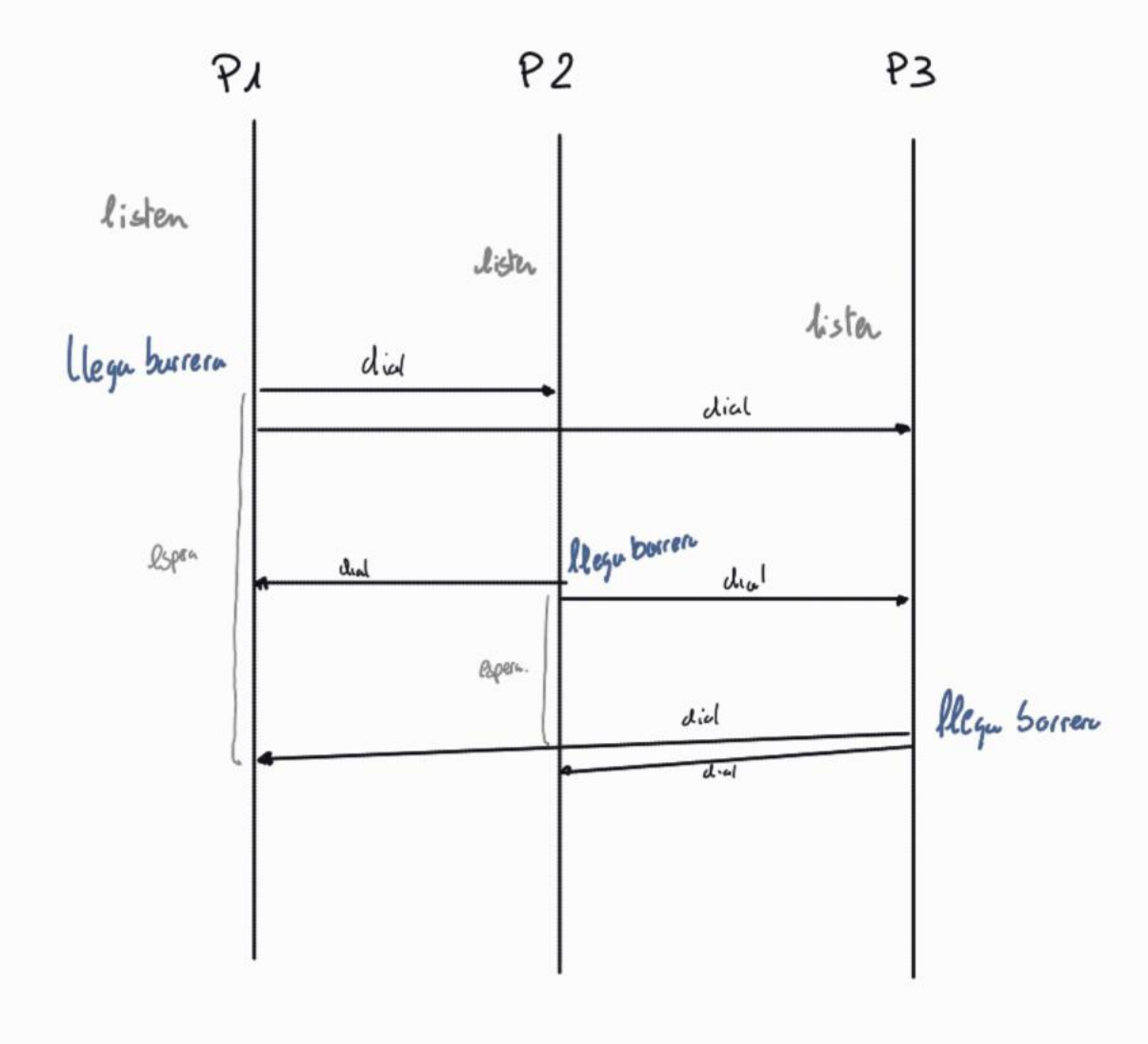
El segundo ejercicio vemos cómo podemos gestionar sincronización entre varios procesos distribuidos con la implementación de un algoritmo de Barrera de Sincronización

El reto es que existen varios procesos ejecutándose en paralelo y tenemos que garantizar que ningún proceso sobrepase un punto de control determinado en su código hasta que el resto de procesos están en ese mismo punto. Lo interesante es que este algoritmo no necesita coordinador

Algoritmo

1. Cada proceso escucha su endpoitn
2. Cuando llega a la barrea cada proceso avisa a los otros N-1, enviándoles su número de línea.
3. Cada proceso espera recibir avisos de N-1 procesos distintos. Al recibirlos, sabe que todos han llegado y supera la barrera.

Diagrama de secuencia



Ejercicio 3

Problema cliente servidor y master workers

En este problema se intenta solucionar un problema complejo como el cálculo de números primos en rangos numéricos elevados mediante concurrencia. Para ello se estudia dos diferentes métodos: cliente-servidor y master-worker

Elementos comunes:

- Los mensajes (com.Request{Id, Interval} y com.Reply{Id, Primes}) son serializados con gob. Cada conexión TCP transporta una petición y su respuesta, por lo que se crea un codificador y un decodificador por conexión.
- Como identificar fin del programa: con una petición con Id = -1
- El cliente es el suministrado, sin modificar

Cliente-servidor

Se puede generar una goroutine por cada petición pero esto tiene un sobrecoste por lo que se ha implementado un conjunto de goroutines (pool) que se crean al arrancar y se reutilizan.

El servidor se comunica con el pool mediante dos canales síncronos: uno de tareas y otro de resultados

El diseño tiene 3 pasos:

- Acceptor (la goroutine main): acepta conexiones, decodifica la petición y la envía por el canal tasks empaquetada con su conexión
- Workers: leen del canal tasks, calculan los primos y escriben en el canal results.
- Responder: lee de results, envía la respuesta por la conexión del cliente y la cierra.

Diseño

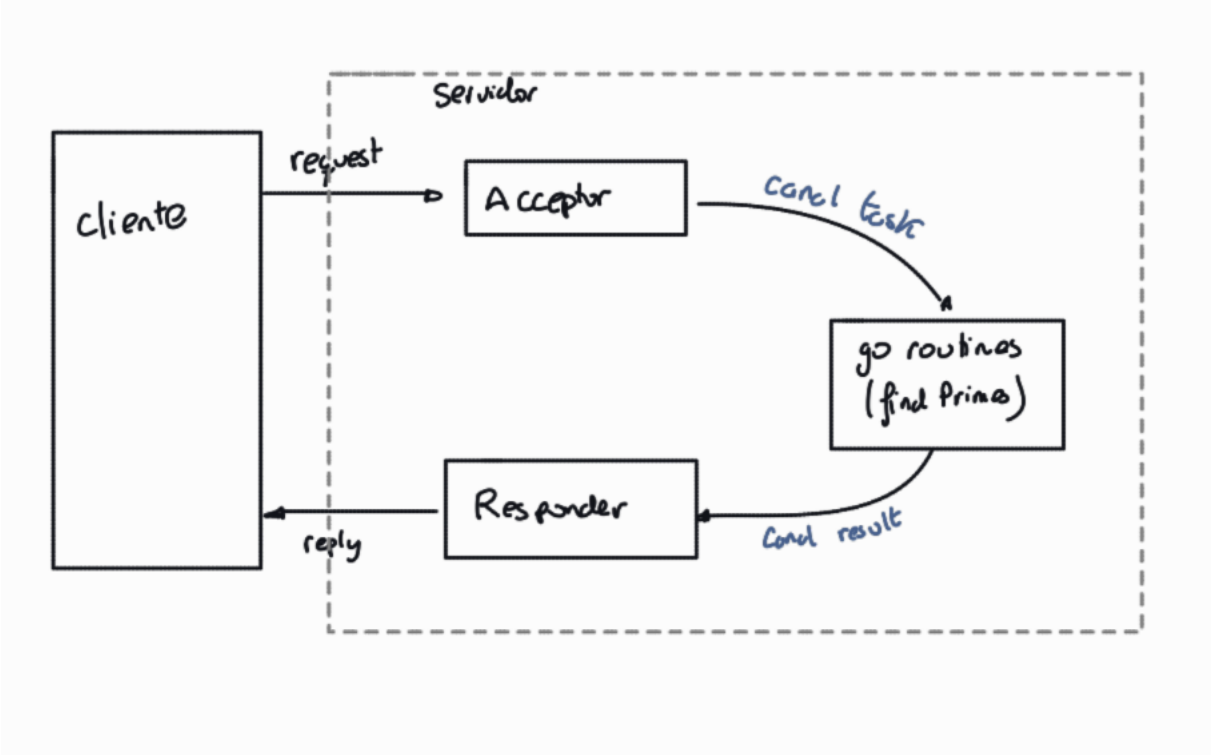
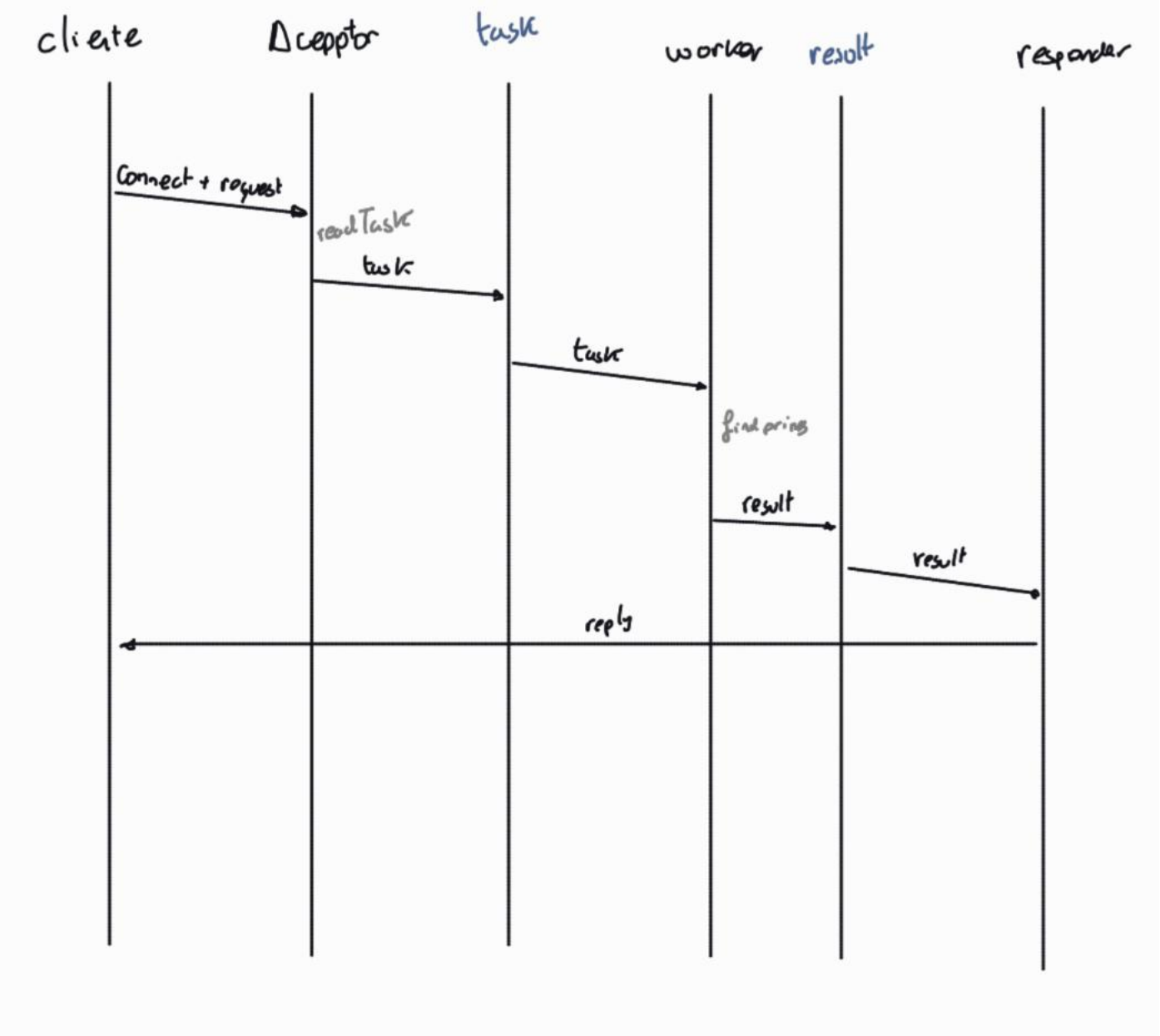


Diagrama de secuencias



Master-worker

En la arquitectura máster-worker el cálculo no se realiza en el proceso que atiende a los clientes, sino en procesos remotos. He reutilizado la arquitectura anterior (acceptor, canal de tareas, canal de resultados y responder). Pero se hace una conexión ssh para calcular

Observaciones:

- Fichero de workers. El máster lee de un fichero una línea ip:puerto por worker. Pueden ponerse varios workers en una misma máquina con puertos distintos.
- Sincronización de arranque. El worker puede tardar en empezar a escuchar, de modo que la goroutine del máster reintenta la conexión hasta 20 veces, con una pausa de 0,5 s a de 0,

Diseño

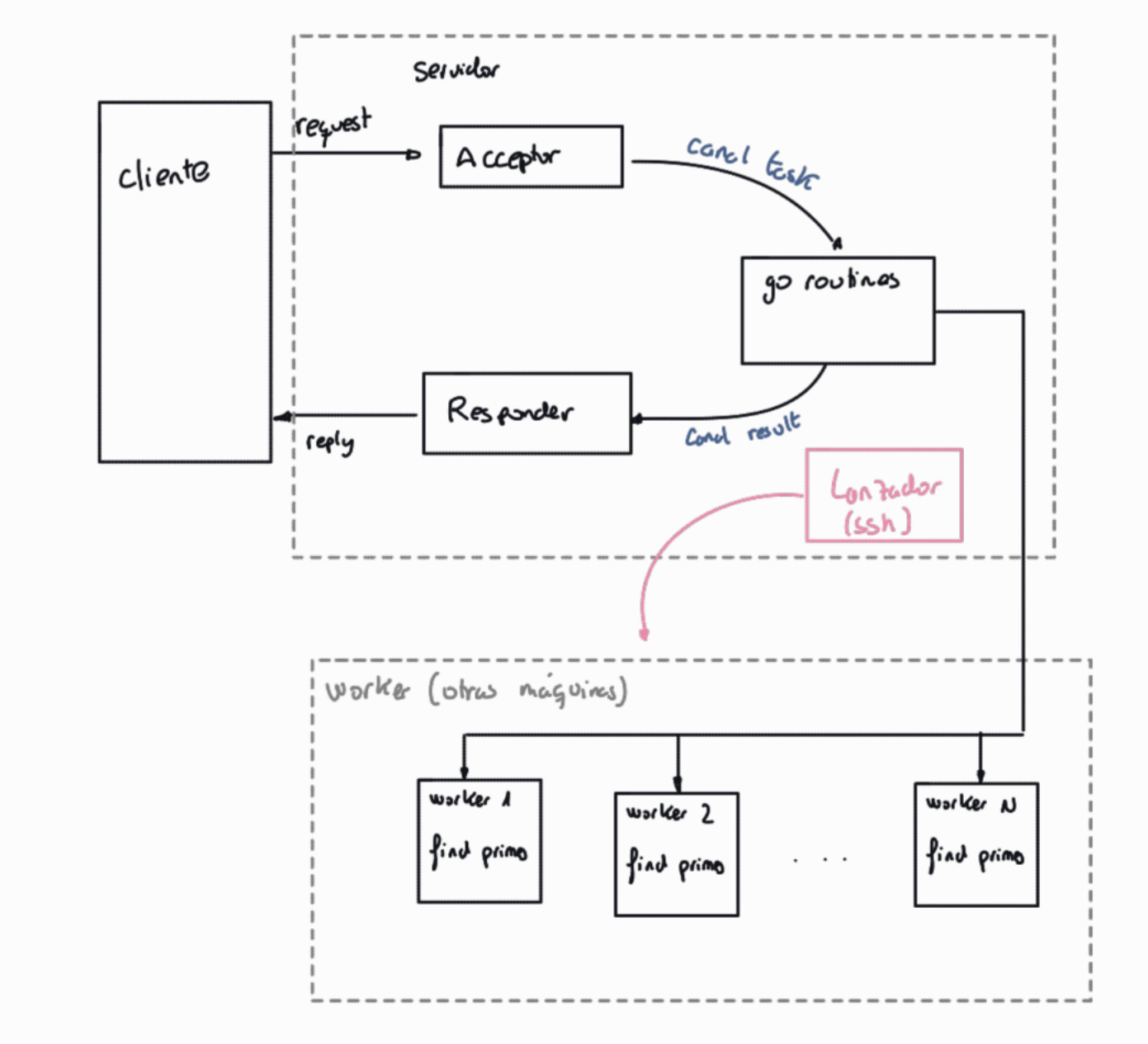
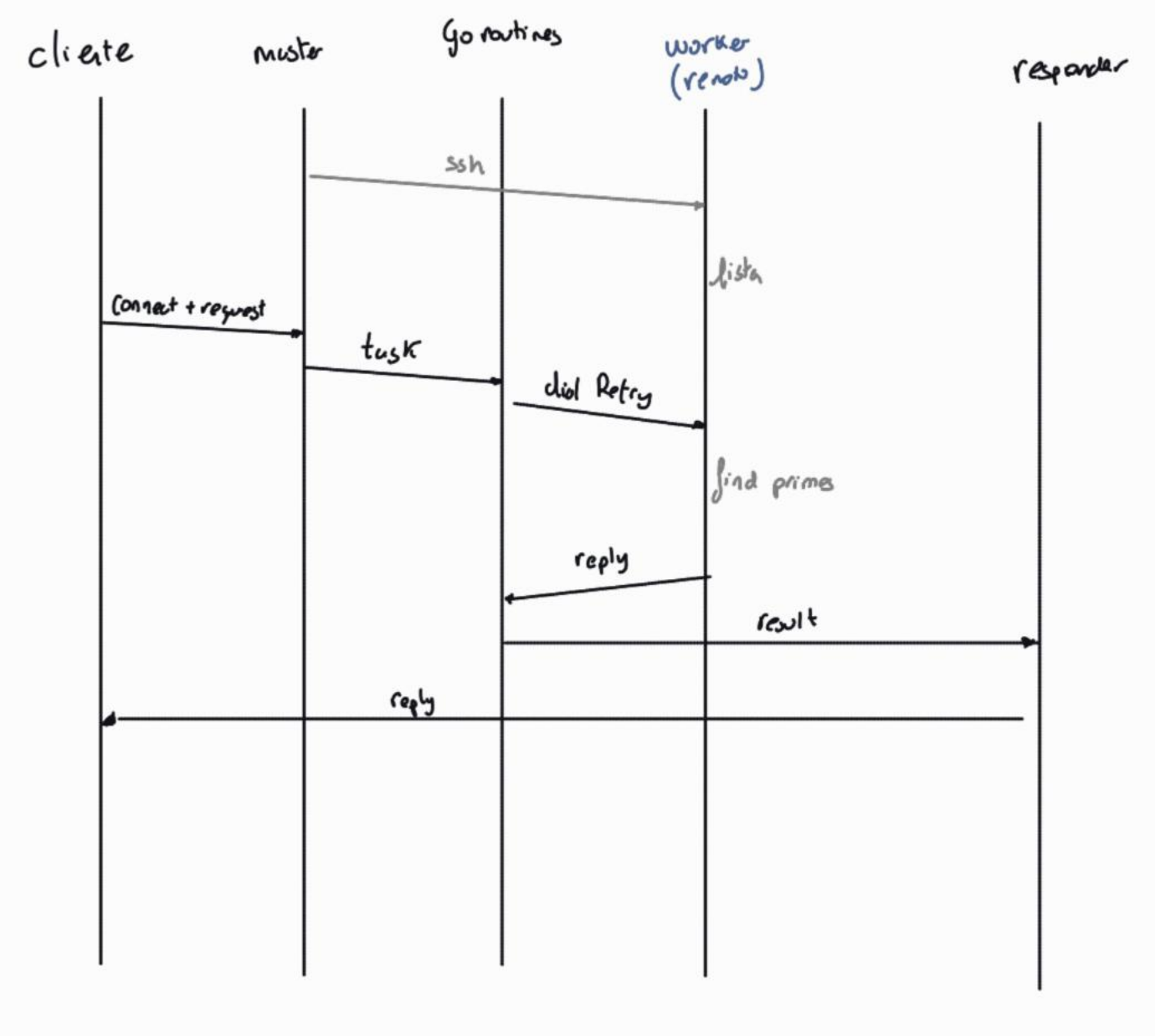

Servidor
cliente
request
Acceptor
Conn task
go routinas
Responder
reply
Conn result
Lanzador
(ssh)
Worker (otras máquinas)
worker 1
find primo
worker 2
find primo
. . .
worker N
find primo

Diagrama de secuencias



Comparación de las dos arquitecturas

Aspecto	**Pool de goroutines**	**Máster-worker**
Dónde se calcula	En el propio servidor, en goroutines	En procesos remotos (otras máquinas)
Paralelismo máximo	Núcleos de una máquina (4)	Número de workers (puede superar una máquina)
Arranque	Un solo proceso	El máster lanza los workers por ssh
Coste de comunicación	Canales en memoria	Conexión TCP y serialización con gob por tarea
Tolerancia a fallos	Ninguna (un solo proceso)	Detecta el fallo; la tarea se pierde